BT MINI PROJECT · FINAL YEAR B.E.

e-Voting dApp using Blockchain

Your mini project explained section by section, with the code walked through and probable viva / oral questions with answers

1	**The project at a glance**	Part 1
2	**Why use a blockchain for voting?**	Part 2
3	**The smart contract explained**	Part 3
4	**Deployment and your test results**	Part 4
5	**Preferential voting with vote transfer**	Part 5
6	**From contract to full dApp, and the security analysis**	Part 6
7	**Probable viva questions with answers**	Part 7

Based on your report **PCCOER_eVoting_dApp_Blockchain_Mini_Project_Report.pdf** and its contract **EVotingSystem** (election.sol).

How to use: read the theory once, trace the worked example by hand, then cover the answers and test yourself on the questions. Answers are written the way you would say them in a viva — short first line, then the detail if the examiner digs deeper.

PART 1

The project at a glance

Title: dApp for e-Voting using Blockchain (Blockchain Technology mini project)
Team: Yash Dhadge (BECOA41), Sarvesh Dharmadhikari (BECOA45), Divyesh Ravane (BECOA62), Vaishnav Pawar (BECOA61)

Problem statement

Develop a blockchain-based decentralised application (dApp) for an e-voting system.

What was built and what was designed

Part	Status in your report
Solidity smart contract **EVotingSystem** — register voters, add parties, one vote per registered voter, read the tally	**implemented**, compiled with Solidity 0.8.21 and tested in **Remix IDE** on **Remix VM (Shanghai)**
Preferential voting with vote transfer — eliminate the last candidate and redistribute votes until a majority set by the organisation is reached	**designed** (described in the report), not yet coded
Full dApp: **MetaMask** wallet, **Truffle** framework, **Ganache** local chain, **Web3.js** front end	**proposed** tools for the complete system

Be clear about this split in the viva — examiners respect an honest "this part is implemented, this part is future scope".

ONE-MINUTE ANSWER TO "EXPLAIN YOUR PROJECT"

"Our project is an e-voting dApp on Ethereum. The voting rules are written in a Solidity smart contract called EVotingSystem. The account that deploys it becomes the admin, who registers voter IDs and adds parties. A registered voter can vote exactly once for a valid party — modifiers enforce that — and anyone can read the vote count of each party. Every action emits an event, so the whole election is auditable on the blockchain and votes cannot be altered afterwards. We deployed and tested it in Remix. Our design also proposes preferential voting with vote transfer, so a second election round is not needed when no candidate has a clear majority."

PART 2

Why use a blockchain for voting?

What a trustworthy election needs

Requirement	Meaning	How a blockchain helps
Eligibility	only registered voters vote	contract checks `isRegistered`
One person, one vote	no double voting	contract checks `hasVoted`
Integrity	votes cannot be changed or deleted	blocks are hash-linked and replicated — immutable
Accuracy	every vote counted as cast	counting is done by code that everyone can verify
Transparency / auditability	anyone can verify the result	public ledger + events
Availability	no single point of failure	thousands of nodes keep a copy
Ballot secrecy	nobody can link a vote to a voter	**not** automatic — public chains are transparent (see Part 6)

Centralised e-voting vs blockchain e-voting

	Centralised server	Blockchain dApp
Who controls the data	one organisation / database admin	no single party; consensus of nodes
Tampering	an insider can edit the database	changing a vote means rewriting the chain — practically impossible
Verification	voters must trust the organiser	anyone can recount from the ledger
Single point of failure	yes	no
Cost of each action	almost free	gas fee per transaction (on a public chain)
Privacy	data hidden but trust required	data public unless extra cryptography is used

Key blockchain terms used in the project

- **Blockchain** — a distributed, append-only ledger of blocks linked by cryptographic hashes.
- **Ethereum** — a programmable blockchain that runs **smart contracts** on the **EVM**.
- **Smart contract** — self-executing code on the blockchain; here it *is* the election officer.
- **dApp** — decentralised application = smart contract (back end) + web front end + wallet.
- **Gas** — fee for every state-changing operation (adding voters, parties, votes).
- **Event** — a log entry in the transaction receipt, used for audit trails and front-end updates.

PART 3

The smart contract explained

Data structures

```
struct Voter { bool isRegistered; bool hasVoted; uint8 votedPartyId; }
struct Party { string name; uint256 voteCount; }

address public admin;                       // deployer; only one who can add voters and parties
Party[] public parties;                     // party ID = position in this array (0, 1, 2 ...)
mapping(string => Voter) public voters;     // voter ID string -> voter record

event VoterRegistered(string voterId);
event PartyAdded(uint256 partyId, string partyName);
event Voted(string voterId, uint256 partyId);
```

- **struct** groups related fields into one type.
- **Dynamic array** `Party[]` grows with `push`; its `length` is the number of parties.
- **mapping** is a key → value table; every key exists with default values (false, false, 0), so an unregistered ID simply has `isRegistered == false`.
- **public** state variables get automatic getters — that is why Remix shows `admin`, `parties(index)` and `voters(id)` buttons.

Constructor and modifiers

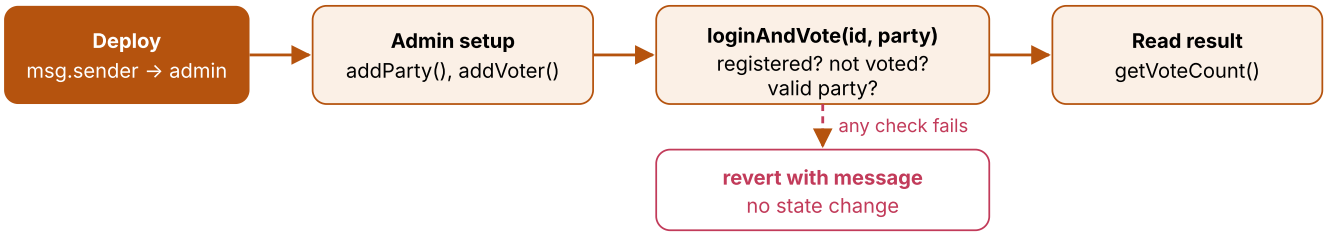
```
constructor() { admin = msg.sender; }

modifier onlyAdmin() {
    require(msg.sender == admin, "Only admin can perform this operation"); _;
}
modifier onlyRegisteredVoter(string memory _voterId) {
    require(voters[_voterId].isRegistered, "Voter is not registered"); _;
}
modifier hasNotVoted(string memory _voterId) {
    require(!voters[_voterId].hasVoted, "Voter has already voted"); _;
}
```

- The **constructor** runs once at deployment; `msg.sender` (the deploying account) becomes the admin.
- A **modifier** is a reusable check run before the function body; `_;` is where the body executes. If a `require` fails, the whole transaction **reverts** with the message and no state changes.

Functions

Function	Who can call	What it does	Gas?
`addVoter(voterId)`	admin only	rejects duplicates, sets `isRegistered = true`, emits `VoterRegistered`	yes (transaction)
`addParty(name)`	admin only	`partyId = parties.length`, pushes `Party(name, 0)`, emits `PartyAdded`	yes
`loginAndVote(voterId, partyId)`	anyone with a registered, unused voter ID	checks valid party, sets `hasVoted = true`, stores `votedPartyId`, increments `voteCount`, emits `Voted`	yes
`getVoteCount(partyId)`	anyone	returns the party's votes (`view`)	free call
`getPartyCount()`	anyone	returns `parties.length` (`view`)	free call
`admin`, `parties(i)`, `voters(id)`	anyone	automatic getters	free calls



Life of an election in EVotingSystem.

Why the order inside loginAndVote matters

All checks (modifiers and `require(_partyId < parties.length)`) run **before** any state is changed — the **Checks → Effects** pattern. `hasVoted` is set to true in the same transaction as the count increment, and a transaction is **atomic** (all or nothing), so there is no way to count a vote without also marking the voter.

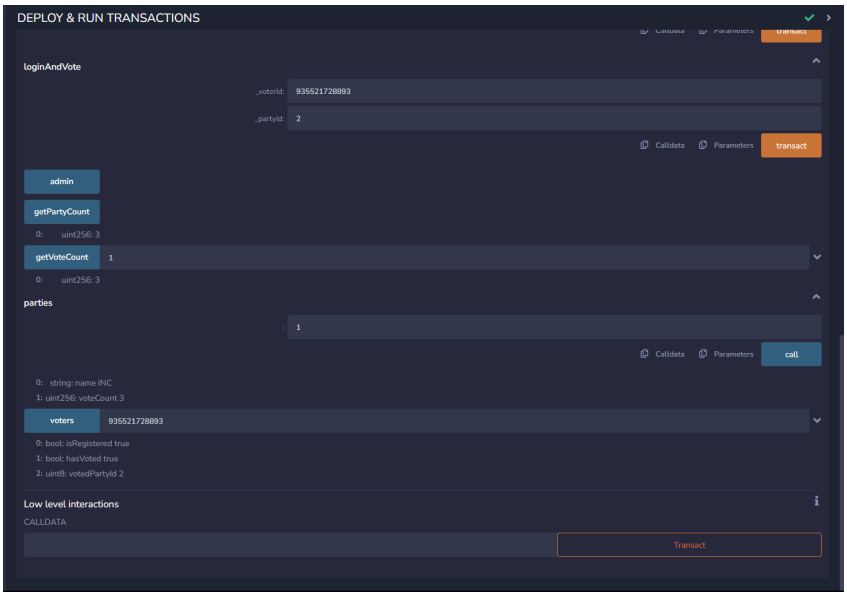
PART 4

Deployment and your test results

Environment: Remix IDE in Chrome, file `election.sol`, Solidity **0.8.21** (optimizer off), **Remix VM (Shanghai)** with a test account of ~100 ETH and a gas limit of 3,000,000.

Remix VM is a blockchain simulated inside the browser — instant, free test ETH, no real network. **Shanghai** is the Ethereum upgrade (2023) whose EVM rules the VM follows.

Call	Input	Result	What it proves
`addParty`	"AAP"	party appended	admin can add parties; ID = array index
`addVoter`	935521728893	voter registered	admin registers eligible voters
`loginAndVote`	935521728893, party 2	vote recorded	a registered voter can vote for a valid party
`getPartyCount`	—	3	three parties exist (IDs 0, 1, 2)
`getVoteCount`	1	3	party 1 has 3 votes
`parties`	1	name INC, voteCount 3	struct getter returns both fields
`voters`	935521728893	isRegistered true, hasVoted true, votedPartyId 2	the voter is marked and their choice stored



Your Remix screenshot after voting: getPartyCount = 3, party 1 = INC with 3 votes, and the voter record showing hasVoted = true and votedPartyId = 2.

Negative tests to demonstrate in the viva (each should revert with its message): a second `loginAndVote` with the same ID → "Voter has already voted"; an unregistered ID → "Voter is not registered"; party ID 99 → "Invalid party ID"; `addVoter` from a non-admin account → "Only admin can perform this operation"; registering the same ID twice → "Voter already registered".

PART 5

Preferential voting with vote transfer

The idea

In normal (**first-past-the-post**) voting each voter picks one candidate and the highest count wins — even with, say, 35% of the votes. If the organisation requires a **majority** (e.g. more than 50%), a fresh second round costs time and money. In **preferential voting** (also called **instant-runoff voting / ranked-choice voting**) each voter **ranks** candidates once, and the runoff happens automatically:

1. Count every ballot for its **first** preference.
2. If a candidate has the required majority, they win.
3. Otherwise **eliminate the candidate with the fewest votes** and transfer each of their ballots to the **next preference** still in the race.
4. Repeat until someone reaches the majority.

Worked example — 100 voters, majority = more than 50

Round	A	B	C	D	Action
1 (first preferences)	40	30	20	**10**	no one > 50 → eliminate D
2	40	34	**26**	—	D's 10 ballots: 4 → B, 6 → C. Still no majority → eliminate C
3	48	**52**	—	—	C's 26 ballots: 18 → B, 8 → A. **B wins**

A led the first count, but B is preferred by a majority once lower preferences are counted — the result reflects the electorate's wishes without a second polling day. The "majority" threshold (50%, 60% …) can be set by the organisation, as your report proposes.

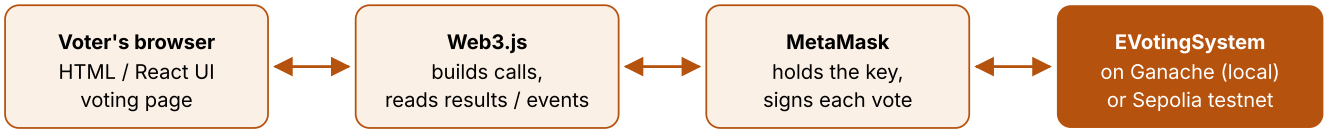
How it could be added to the contract

- Store each voter's **ranking** as an array, e.g. `uint8[] preferences` (party IDs in order), instead of one `votedPartyId`.
- Add a `tally()` function that loops over rounds: count the highest-ranked still-active preference of each ballot, check the majority, mark the lowest party eliminated, repeat.
- **Gas concern:** looping over all ballots for every round costs gas proportional to voters × rounds, and a block has a gas limit. For large elections, either count **off-chain** from the on-chain ballots and events (anyone can verify the result), or count in batches.

PART 6

From contract to full dApp, and the security analysis

Full dApp architecture (as proposed)



Truffle (or Hardhat) compiles, tests and deploys the contract; Ganache provides a local blockchain with test accounts

Tool	Role
Truffle	framework to compile (`truffle compile`), deploy with migration scripts (`truffle migrate`) and test (`truffle test`) contracts
Ganache	personal local Ethereum blockchain with 10 funded test accounts; instant blocks for development
MetaMask	wallet that holds each voter's private key and signs their transactions; connects the web page to the chain
Web3.js	JavaScript library the front end uses to call contract functions through MetaMask and listen to events (ethers.js is a popular alternative)

Note for the viva: ConsenSys **sunset Truffle and Ganache in 2023** (your reference already points to the archived docs). Today most projects use **Hardhat** or **Foundry**; the concepts are the same.

Security analysis — limitations of the current contract

These are the questions a sharp examiner will ask. Knowing them shows you understand your own design.

Issue	Why it matters	Fix
Voter ID not tied to a wallet	`loginAndVote` never checks `msg.sender`; anyone who knows a registered ID can vote in that voter's name	key voters by address: `mapping(address => Voter)` and check `msg.sender`; admin registers addresses
IDs are public	the 12-digit voter ID is stored and emitted in plain text — readable by everyone, forever	store `keccak256` hashes of IDs (with a secret salt), never raw identity numbers
No ballot secrecy	`voters(id)` shows `votedPartyId`; anyone can see who voted for whom → enables coercion and vote-buying	**commit–reveal** (submit hash(vote + salt), reveal later) or zero-knowledge schemes (e.g. MACI, Semaphore)
Central admin	the admin decides who may vote and could register fake voters	multi-signature admin, or a registration authority whose actions are audited through events
No election phases	votes can be cast at any time; results are live during voting	an `enum` state (Registration, Voting, Ended) with `startVoting()` / `endVoting()`
`uint8(_partyId)` cast	explicit conversion silently truncates; with more than 256 parties, ID 256 would be stored as 0	use `uint256` for the stored ID
No winner function	results must be read party by party	add `getWinner()` or the preferential `tally()`
Gas cost & scalability	every vote is a paid transaction; Ethereum mainnet handles ~15–30 tx/s	private / consortium chain or a Layer-2 rollup for large elections

PART 7

Probable viva questions with answers

About the project

Q1. What is the aim of your project?

To build a decentralised e-voting application on Ethereum in which a smart contract enforces voter eligibility and one-vote-per-voter, and records every vote immutably and transparently on the blockchain.

Q2. What exactly did you implement and what is future scope?

Implemented: the EVotingSystem Solidity contract (admin, add voters, add parties, one vote per voter, vote counts, events), deployed and tested in Remix VM. Future scope: preferential voting with vote transfer, a Web3.js front end with MetaMask, Truffle/Ganache deployment, and stronger identity and privacy.

Q3. Why did you choose blockchain for voting?

It removes the need to trust a single organiser: votes are stored on a tamper-evident ledger replicated across nodes, the counting rules are open code, and anyone can audit the result.

Q4. What is a dApp? Why is your project a dApp?

A decentralised application whose back-end logic runs in smart contracts on a blockchain, with a front end that connects through a wallet. Our voting logic runs in the EVotingSystem contract on Ethereum, not on a central server.

Q5. Does your project use a dataset?

No external dataset. The data is the contract's on-chain state: voters (mapping), parties (array) and vote counts, plus the event logs.

Smart contract

Q6. Explain the structure of your smart contract.

Two structs (Voter, Party), three state variables (admin, parties array, voters mapping), three events, a constructor that sets the admin, three modifiers (onlyAdmin, onlyRegisteredVoter, hasNotVoted), two admin functions (addVoter, addParty), the voting function loginAndVote and two view functions (getVoteCount, getPartyCount).

Q7. How do you prevent a voter from voting twice?

The `hasNotVoted` modifier requires `voters[id].hasVoted == false`. The first successful vote sets it to true in the same atomic transaction, so a second attempt reverts with "Voter has already voted".

Q8. How do you make sure only eligible voters vote?

The admin registers each voter ID with addVoter, setting `isRegistered = true`; the `onlyRegisteredVoter` modifier rejects any ID that was not registered.

Q9. Who is the admin and how is that decided?

The account that deployed the contract — the constructor stores `msg.sender` in `admin`. The `onlyAdmin` modifier allows only that address to add voters and parties.

Q10. What is a modifier? Explain `_;`.

A reusable condition attached to functions, run before the body. `_;` marks where the function body is inserted. If the modifier's `require` fails, the function never runs.

Q11. What does `require` do when it fails?

It stops execution, reverts every state change made in the transaction, returns the error message, and refunds the unused gas (the gas already used is still charged).

Q12. Why use a mapping for voters and an array for parties?

Voters are looked up by ID, and a mapping gives constant-time lookup without iterating. Parties need to be counted and listed in order, so an array (with `length` and index-based IDs) fits better. Mappings cannot be iterated or report their size.

Q13. How is a party ID assigned?

`partyId = parties.length` before the push, so the first party is 0, the second 1, and so on — the ID is the party's index in the array.

Q14. What are events and why did you use them?

Events write logs to the transaction receipt: VoterRegistered, PartyAdded and Voted. They are much cheaper than storage, create an auditable trail of every action, and let a front end react to new votes.

Q15. Which functions cost gas and which are free?

addVoter, addParty and loginAndVote change state, so they are transactions that cost gas. getVoteCount, getPartyCount and the public getters are `view` calls, free when called from outside.

Q16. What is the difference between a transaction and a call?

A transaction is signed, changes state, is included in a block and costs gas. A call only reads data from a node, changes nothing and is free.

Q17. What is `msg.sender`?

The address that sent the current transaction or call. In the constructor it is the deployer; in onlyAdmin it is compared with the stored admin.

Q18. Why did you use `memory` for string parameters?

Strings are reference types, so their data location must be stated. `memory` holds a temporary copy for the duration of the call. (For external functions `calldata` would be slightly cheaper.)

Q19. Can a vote be changed or deleted after it is cast?

No. The contract has no function to change a vote, and the blockchain record of the transaction is immutable.

Q20. Can the contract code be changed after deployment?

No, deployed bytecode is immutable. To fix or extend it we must deploy a new contract (or design an upgradeable proxy from the start).

Testing and tools

Q21. How did you deploy and test the contract?

In Remix IDE: compiled election.sol with Solidity 0.8.21, deployed to Remix VM (Shanghai) from a test account with 100 ETH, then called addParty, addVoter and loginAndVote and read getPartyCount, getVoteCount, parties and voters to confirm the state.

Q22. What is Remix VM?

A simulated Ethereum blockchain that runs inside the browser in Remix, with pre-funded test accounts and instant blocks. Nothing is sent to a real network.

Q23. Explain your test results.

After the test, there were 3 parties; party 1 (INC) had 3 votes; voter 935521728893 showed isRegistered = true, hasVoted = true and votedPartyId = 2 — confirming registration, voting and recording all worked.

Q24. What is Ganache? What is Truffle?

Ganache is a personal local Ethereum blockchain for development with funded test accounts. Truffle is a framework to compile, migrate (deploy) and test smart contracts. Both were sunset by ConsenSys in 2023; Hardhat and Foundry are the current alternatives.

Q25. What is Web3.js?

A JavaScript library that lets a web page talk to Ethereum nodes — send transactions through MetaMask, call view functions and subscribe to events.

Q26. What is the role of MetaMask in the full dApp?

It stores each voter's private key, shows the transaction and fee, signs the vote transaction and sends it to the network. It also identifies the voter by their address.

Q27. What is gas and who pays it in your system?

Gas measures computation on Ethereum; every state-changing call needs a fee. The admin pays for adding voters and parties, and each voter pays for their own vote — in a real election the organiser might sponsor fees (meta-transactions or a private chain with free gas).

Preferential voting

Q28. What is preferential voting with vote transfer?

Voters rank candidates. If nobody has the required majority of first preferences, the last-placed candidate is eliminated and their ballots move to the next preference. This repeats until someone has the majority — an automatic runoff (instant-runoff voting).

Q29. Why is it better than a normal majority vote?

It avoids a costly second election round when no candidate has a clear majority, and the winner is acceptable to a majority of voters rather than just the largest minority.

Q30. How would you implement it in Solidity?

Store a ranked array of party IDs per voter and write a tally function that, round by round, counts each ballot's top non-eliminated preference, checks the majority threshold, and eliminates the lowest party. For large elections the loops would be costly, so counting could be done off-chain from the on-chain ballots and verified by anyone.

Security, privacy and limitations

Q31. Is your voting system secure? What are its weaknesses?

It is tamper-proof and prevents double voting by ID, but: the voter ID is not linked to a wallet, so anyone who knows an ID could vote for that person; IDs and choices are public, so there is no ballot secrecy; the admin is a central point of trust; and there are no start/end phases.

Q32. How would you link a vote to a real voter securely?

Register each voter's wallet address (after off-chain identity verification such as KYC or biometrics) and check `msg.sender` in the voting function. Then only the holder of that private key can vote.

Q33. How can votes be kept secret on a public blockchain?

With a commit–reveal scheme (submit a hash of vote + secret salt, reveal after voting closes), homomorphic encryption of tallies, or zero-knowledge proofs (e.g. Semaphore, MACI) that prove "a registered voter voted once" without revealing who.

Q34. What is the commit–reveal scheme?

In the commit phase each voter submits keccak256(vote, salt); nobody can see the vote. In the reveal phase the voter submits the vote and salt; the contract checks the hash and counts it. It hides votes during the election, but not after reveal unless combined with anonymity techniques.

Q35. Why is storing a 12-digit ID on chain a problem?

Everything on a public blockchain is visible forever. Identity numbers are personal data; publishing them breaks privacy laws and allows impersonation. Store only salted hashes or use wallet addresses.

Q36. What could go wrong with `uint8(_partyId)`?

An explicit conversion to uint8 does not revert — it keeps only the lowest 8 bits. With more than 256 parties, party 256 would be recorded as 0. Using uint256 avoids it.

Q37. Can the admin cheat?

The admin cannot change or delete votes, but could register fake voter IDs. Mitigations: publish the registration list (events already help), multi-signature admin control, or an independent registration authority.

Q38. What is a 51% attack and does it threaten your election?

If an attacker controls most of the mining power or stake, they can reorder or reverse recent transactions. On a large public chain like Ethereum it is extremely expensive; on a small private chain the validator set must be trustworthy.

Q39. What are the challenges of blockchain voting in real elections?

Scalability (crores of voters vs ~15–30 transactions per second on Ethereum L1), gas costs, voter privacy and coercion resistance, secure identity verification, accessibility for voters without smartphones or wallets, key loss, and legal acceptance.

Q40. Which type of blockchain would you use for a national election?

A permissioned (consortium) blockchain run by the election commission, courts and independent observers — fast, no per-vote fees and controlled membership — with results or hashes anchored to a public chain for independent verification (a hybrid approach).

Q41. What did you learn from this project?

How to model real rules (eligibility, one vote) as smart-contract code with modifiers and require checks, how events create an audit trail, how to deploy and test in Remix, and that transparency, privacy and identity must be designed together for voting.